Detailed DOC -Pretinkya

Server Configuration and Initialization

Imports and Initialization (server.js)

Environment variable loading — Loaded environment variables from .env file using dotenv for configuration management

Express framework setup — Initialized Express web framework for handling HTTP requests and responses

HTTPS module import — Imported Node.js HTTPS module for secure server communication

File system module import — Imported fs module for reading and writing files including SSL certificates

Path module import — Imported path module for constructing cross-platform file paths

Session management import — Imported express-session for user session handling and persistence

Database connection import — Imported database module for MySQL connection and query execution

Admin API routes import — Imported adminRoutes module containing all administrative endpoint handlers

Data export routes import — Imported dataExportRoutes module for exporting feedback and archive data

OS utilities import — Imported os module for accessing network interface information

Data retention cleanup import — Imported dataRetentionCleanup module for automatic archive management

QR code generation import — Imported QRCode library for generating access QR codes

Express app instantiation — Created Express application instance for configuring middleware and routes

Server port definition — Set default port to 3000 for server binding

Network Interface Functions (server.js)

Network IP enumeration — Implemented getAllNetworkIPs() to discover all available IPv4 network interfaces with their addresses, MAC addresses, and CIDR notation

IP selection logic — Implemented getSelectedIP() with priority hierarchy: command line argument, environment variable, then first available IP

IP validation — Validated user-selected IP addresses against available network interfaces with fallback to first available IP

Interface name lookup — Implemented getInterfaceForIP() to map IP addresses to their corresponding network interface names

Local IP determination — Selected and stored the active IP address for server binding

Interface name storage — Stored the network interface name corresponding to the selected IP

SSL Certificate Configuration (server.js)

SSL certificates directory — Defined certsDir path pointing to certs/ folder for SSL certificate storage

Certificate file path — Defined certPath pointing to selfsigned.pem in certs/ directory

Key file path — Defined keyPath pointing to selfsigned.key in certs/ directory

SSL options storage — Declared sslOptions variable to hold certificate and key for HTTPS server creation

Middleware Configuration (server.js)

JSON body parsing — Configured express.json() middleware to parse incoming JSON request bodies

URL-encoded body parsing — Configured express.urlencoded() with extended option for parsing form data

Session middleware — Configured express-session with secret from environment variable, cookie settings (httpOnly, secure based on HTTPS, sameSite), resave and saveUninitialized flags

Static file serving — Configured express.static() to serve frontend files from ../frontend directory

API Routes (server.js)

Admin API routing — Mounted adminRoutes at /api/admin for all administrative operations

Data export routing — Mounted dataExportRoutes at /api/admin/data-export for feedback and archive export functionality

Network interfaces endpoint — Implemented GET /api/network-interfaces to return all available network interfaces as JSON

Server info endpoint — Implemented GET /api/server-info to return selected IP, protocol type, and interface information

QR code generation endpoint — Implemented GET /api/generate-qr to generate QR codes for feedback page access with customizable parameters

Database test endpoint — Implemented GET /api/test-db to verify MySQL connection and list all tables in the database

Page Routes (server.js)

Admin page route — Configured GET /admin to serve admin.html from frontend/admin directory

Certificate and Server Functions (server.js)

Server startup function — Implemented startServer() to create HTTPS server if SSL options available, otherwise HTTP server, binding to selected IP and port

Server information display — Implemented printServerInfo() to output formatted console information including protocol, URLs for feedback/tree/admin/data-export, timestamp in Singapore timezone, database name, SSL certificate location, available network interfaces with current selection indicator, and QR code access information

Server Startup and Initialization (server.js)

Help command handling — Implemented command-line help display showing usage, options (--ip flag), environment variables, and available network IPs, exiting after display

Server execution — Called startServer() to launch the HTTP or HTTPS server with all configured routes and middleware

---

FEEDBACK

Feedback Page Styling and Layout

Landing Page Styles (feedback.css)

Right section QR panel — Styled right section container with semi-transparent white background, rounded corners, centered text alignment, border, and shadow for QR code display area

QR Code Section (feedback.css)

QR header and icon — Styled header section with centered flexbox layout, icon spacing, color scheme, and typography for QR code section title

QR code frame and image — Styled white background container with padding, rounded corners, inline-block display, shadow, and fixed 200 x 200 pixel dimensions for QR code image display

QR description text — Styled descriptive text below QR code with color, font size, and line height for instruction display

Data Retention Consent Page (feedback.css)

Consent card and header — Styled centered consent page container with white background, rounded corners, 600px width, centered layout, shadow, border, and flex column layout for vertical content arrangement

Retention options grid — Styled two-column grid layout with 20px gap for displaying data retention choice cards

Privacy notice — Styled information banner with light background, left-aligned text, left border accent, icon spacing, and padding for displaying privacy information

Choose Style Page (feedback.css)

Style card layout — Styled style selection page container with white background, rounded corners, 900px width, centered layout, padding, shadow, and border

Theme preview frame — Styled preview container with 800px width, 450px height, rounded corners, centered positioning, overflow hidden, shadow, and white background for displaying photo with overlay

Theme grid and selection state — Styled responsive auto-fit grid for theme cards, hover effects with border color change and upward translation, selected state with green background and border, animated top accent bar, and visual feedback

Selected theme display — Styled confirmation banner with green background, padding, rounded corners, green border, centered text, and green text color to show currently selected theme

Button Components (feedback.css)

Consent button with disabled state — Styled full-width button with gradient background, white text, rounded corners, shadow, shimmer animation effect, hover lift effect, and disabled state with reduced opacity and no-interaction cursor

Device Detection and Overlay Styles (feedback.css)

Desktop vs mobile preview sizing — Applied device-specific dimensions to preview frame with 800› 450 pixels for desktop and 324› 405 pixels for mobile devices

Overlay container and layering — Styled overlay composition container with relative positioning, full dimensions, rounded corners, overflow hidden, shadow, and white background with z-index layering for photo background (z-index 1) and overlay image (z-index 2)

Unified preview background — Applied white background color to all preview frames on style page to ensure consistent display

Feedback Page HTML Structure

QR Code Display (feedback.html)

Dynamic QR code insertion — Created container div with id "dynamic-qr" where server-generated QR code SVG is dynamically inserted via JavaScript to display actual server URL

Fallback static QR code — Provided static SVG QR code placeholder with position markers and data pattern as fallback when dynamic QR code loading fails

Data Retention Consent Page (feedback.html)

Data retention consent page — Created complete consent page section with hidden display state, card container, lock icon, title and description, two-column grid of retention options (7 days and indefinite), privacy notice banner with icon and text, and disabled proceed button that enables on option selection

Feedback Questions (feedback.html)

Database-driven questions container — Created empty questions-container div where feedback questions are dynamically loaded from database and rendered as form elements via JavaScript

Dynamic question loading — Implemented placeholder for questions that are fetched from API and inserted as interactive form fields with ratings, text inputs, or combined question types

Style Selection Page (feedback.html)

Style selection page — Created complete overlay selection page with hidden display state, style card container, gradient icon, title and description, preview frame showing photo with overlay composite, themes section with heading, empty overlay-options grid container for dynamic overlay loading, selected theme display banner, and three action buttons (continue, back, retake)

Dynamic overlay options — Created empty overlay-options grid container where available overlay themes are dynamically loaded from database and rendered as selectable theme cards with swatches and names

Feedback Collection Client-Side Logic

Global Variables and Constants (feedback.js)

Retention option storage — Declared selectedRetention variable to store user's chosen data retention preference (indefinite or 7 days)

Theme selection storage — Declared selectedTheme variable initialized to 'nature' to track currently selected overlay theme

User data object — Declared userData object to store all user input including name, email, phone, answers, and pledge text

Photo data storage — Declared photoData variable to store base64-encoded photo data captured or uploaded by user

Device type detection — Declared currentDevice variable to identify if user is on 'desktop' or 'mobile' for conditional UI behavior

Inactivity timer reference — Declared inactivityTimer variable to hold the timeout reference for user inactivity tracking

Inactivity timeout constant — Defined INACTIVITY_TIMEOUT constant as 300,000 milliseconds (5 minutes) for automatic session timeout

Initialization and Setup Functions (feedback.js)

Dynamic QR code loading — Implemented loadDynamicQRCode() to fetch QR code SVG from server API and replace static QR code with dynamic one showing actual server URL

Device type detection — Implemented detectDeviceType() to identify mobile or desktop devices using user agent detection and apply device-specific CSS classes to body element

Inactivity Timer Functions (feedback.js)

Inactivity timer start — Implemented startInactivityTimer() to begin 5-minute countdown that redirects to landing page when user is inactive

Inactivity timer reset — Implemented resetInactivityTimer() to clear and restart the inactivity countdown on any user interaction

Landing page return — Implemented returnToLandingPage() to reset all form data, clear selections, stop camera stream, and return to initial landing page when timeout occurs

Timeout notification display — Implemented showTimeoutNotification() to create and show a modal notification informing user of session timeout before redirecting

Question Management Functions (feedback.js)

Feedback questions loading — Implemented loadFeedbackQuestions() to fetch active questions from database API and populate form dynamically

Feedback form update — Implemented updateFeedbackForm() to generate question UI elements based on loaded questions and insert into form container

Question element creation — Implemented createQuestionElement() to generate HTML structure for individual questions with rating stars, text input, or no-input types

Question rating selection — Implemented selectQuestionRating() to handle star rating clicks, update visual selection, and store answer in userData object

Question event listeners initialization — Implemented initializeQuestionEventListeners() to attach click handlers to rating stars and input listeners for text questions

No questions message display — Implemented showNoQuestionsMessage() to show informative message when no active questions are available in database

Question type determination — Implemented getQuestionType() to analyze question structure and determine if it's rating-only, text-only, or combined type

Required questions validation — Implemented validateRequiredQuestions() to check all required questions have answers before allowing form submission

Form Submission Functions (feedback.js)

Feedback form submission — Implemented submitFeedback() to validate required questions, store answers in userData, and navigate to pledge page

Photo Handling Functions (feedback.js)

Style page navigation — Implemented continueToStyle() to transition from photo capture/upload page to overlay style selection page

Original photo saving — Implemented saveOriginalPhoto() to send unprocessed base64 photo data to server for storage as original backup

Processed photo saving — Implemented saveProcessedPhoto() to send overlay-applied photo to server for storage as final output

Overlay and Theme Functions (feedback.js)

Overlay options loading — Implemented loadOverlayOptions() to fetch available overlay themes from database API and populate theme selection UI

Default overlay options — Implemented loadDefaultOverlayOptions() to provide fallback overlay themes when database loading fails

Theme color generation — Implemented generateColorFromThemeId() to create consistent background colors for theme cards using theme ID hashing

Theme selection handler — Implemented selectTheme() to handle theme card clicks, update selected state, store choice, and refresh preview image

Theme preview update — Implemented updateThemePreview() to display selected overlay on captured photo in preview area

Cutout positioning preview — Implemented updatePreviewWithCutout() to position user photo within overlay cutout area for accurate preview

Final photo processing — Implemented processFinalPhoto() to composite user photo with selected overlay using canvas operations and generate final image

Page Navigation Functions (feedback.js)

Retention option selection — Implemented selectOption() to handle retention choice clicks (indefinite or 7 days), update UI selection state, and enable proceed button

Style retake navigation — Implemented retakePhotoFromStyle() to navigate back from style page to photo capture/upload page and clear current photo data

Style confirmation — Implemented confirmStyle() to process final photo with overlay, populate confirmation page details, and navigate to confirmation view

Final submission handler — Implemented finalSubmit() to clear inactivity timer, save both original and processed photos, submit all feedback data to server, and show thank you page

Back Navigation Functions (feedback.js)

Consent to landing navigation — Implemented goBackToLanding() to hide consent page, show landing page, and reset inactivity timer

Style to photo navigation — Implemented goBackToPhoto() to hide style page, show appropriate photo page based on device type, reinitialize camera for desktop, and reset inactivity timer

Event Listeners and Cleanup (feedback.js)

Page unload cleanup — Registered beforeunload event listener to stop camera stream and clear inactivity timer when user leaves page

Click interaction listener — Registered click event listener on document to reset inactivity timer on any user click

Keypress interaction listener — Registered keypress event listener on document to reset inactivity timer on any keyboard input

Mouse movement listener — Registered mousemove event listener on document to reset inactivity timer on any mouse movement

Touch interaction listener — Registered touchstart event listener on document to reset inactivity timer on any touch interaction

Feedback API Routes and Logic

Directory Setup (feedbackRoutes.js)

Raw photos directory configuration — Defined uploadsDir constant pointing to uploads/photos directory for storing original unprocessed user photos with recursive directory creation

Processed photos directory configuration — Defined processedDir constant pointing to uploads/processed directory for storing overlay-applied photos with recursive directory creation

Question Management Routes (feedbackRoutes.js)

Active questions retrieval — Implemented GET /questions endpoint to fetch all active questions with their options from database, joining questions and question_options tables, grouping by question ID, and returning structured questions array with options for feedback form rendering

Photo Upload Routes (feedbackRoutes.js)

Raw photo upload — Implemented POST /save-photo endpoint to receive base64 photo data, convert to buffer, generate filename with username, device, and timestamp, save to uploads directory, and return photo metadata including filename and filepath

Processed photo upload — Implemented POST /save-processed-photo endpoint to receive base64 processed photo with overlay, convert to buffer, generate filename including theme name, save to processed directory, and return processed photo metadata

Feedback Submission Routes (feedbackRoutes.js)

Complete feedback submission — Implemented POST /submit-feedback endpoint with optimized fast response strategy (under 2 seconds) that immediately responds to user while processing database save and email sending in background using setTimeout, saves feedback with retention settings and encrypted email, and queues thank you email if valid email and photos exist

Database Operations (feedbackRoutes.js)

Email format validation — Implemented isValidEmail() function to validate email addresses using regex pattern checking for valid email structure

Feedback database save — Implemented saveFeedbackToDatabase() function to orchestrate complete feedback save operation including user lookup by decrypted email comparison, new user creation with encrypted email, existing user update with visit count increment, feedback record insertion with retention settings, and question answers saving

Feedback record save — Implemented saveFeedbackRecord() nested function to insert main feedback record with user ID, pledge text, metadata JSON containing device/theme/retention/photos, photo paths, data retention period, and timestamp

Question answers save — Implemented saveQuestionAnswers() nested function to fetch active questions from database, extract answers from userData object by question ID, insert individual answer records into question_answers table with user ID, feedback ID, question ID, and answer text, and use Promise.allSettled for parallel processing

Feedback Management Routes (feedbackRoutes.js)

All feedback retrieval — Implemented GET /feedback endpoint to fetch all active not_archived feedback records joined with users table, including encrypted email, visits, pledge, retention, photo paths, timestamps, email status, and ordered by creation date descending

All visitors retrieval — Implemented GET /visitors endpoint to fetch all user records with encrypted emails, visit counts, creation dates, and last visit timestamps ordered by most recent visit

Feedback soft deletion — Implemented DELETE /feedback/:id endpoint to perform soft delete by setting is_active flag to 0 rather than removing record from database

Feedback update — Implemented PUT /feedback/:id endpoint to allow admin editing of pledge comment text and admin notes fields for specific feedback record

Utility Endpoints (feedbackRoutes.js)

Database connection test — Implemented GET /test-db endpoint to verify database connectivity by querying information schema for table existence

Root endpoint verification — Implemented GET / endpoint to return success message and list of available feedback API endpoints for route verification

---

ADMIN

Admin Panel Styling and Layout

General Reset and Base Styles (admin.css)

Box sizing and global reset — Applied universal box-sizing border-box and reset margin and padding to zero for all elements

Body and typography base styles — Configured body element with system font stack, theme-based background color variable, and theme-based text color variable

CSS Custom Properties (admin.css)

Global theme color variables — Defined CSS custom properties for themeable colors including background, text, card backgrounds, borders, headers, buttons, navbar, and admin page-specific colors

Semantic color definitions — Defined semantic color variables for success (green), warning (orange), error (red), info (blue), muted and gray tones, overlay backgrounds, and modal backgrounds

Gradient definitions — Defined primary gradient variable with purple-to-pink linear gradient for consistent styling

Login Page Styles (admin.css)

Login container and card — Styled login page with full viewport height flexbox centering, white background, card container with gray background, rounded corners, generous padding, shadow, and centered text

Login form elements and buttons — Styled form groups with labels, text inputs with borders and focus states including box-shadow, and login button with white background, hover effects including lift and shadow

Test accounts display — Styled test accounts section with light background, rounded corners, padding, left-aligned text, and compact typography for displaying development credentials

Admin Header and Layout (admin.css)

Header container and branding — Styled admin header with theme-based background, bottom border, padding, flexbox layout for space-between alignment, and header left section with title and subtitle styling

User info and logout functionality — Styled user info section with flexbox layout, user details column with username and role text, circular avatar with background and centered text, and logout button with dark background and hover effects

Main content area structure — Configured admin content container with flexbox layout and calculated minimum height accounting for header

Sidebar Navigation (admin.css)

Sidebar container and sections — Styled sidebar with fixed 250px width, theme-based background, right border, padding, section spacing, and uppercase section headers with muted color

Navigation items and active states — Styled nav items with flexbox layout, gap, padding, color, hover background change, active state with gradient background and left border accent, and icon sizing

Page Content Area (admin.css)

Page container and active state — Configured page content area with flex-grow, padding, overflow scroll, and display toggling based on active class

Page header styling — Styled page headers with bottom margin, large title font size and weight, and subtitle with muted color

Dashboard Page Styles (admin.css)

Stats cards grid and variations — Styled stats grid with auto-fit responsive columns, card containers with theme backgrounds and borders, colored top accent bars for six color variants (purple, blue, cyan, yellow, green, pink), hover lift effects with variant-specific shadows, and stat content with icon, large value text with gradient, and label

Activity and status sections — Styled activity section containers with theme background, rounded corners, padding, border, and subtle shadow

Section headers and refresh button — Styled section headers with flexbox space-between layout, bottom margin, and padding

Feedback and Archive Data Page Styles (admin.css)

Toolbar with search and filters — Styled toolbar section with search inputs, filter controls, and action buttons in organized layout

Advanced filter panel with animations — Styled collapsible advanced filter panel with smooth height transitions, organized filter groups, and toggle animations

Sortable table headers and results display — Styled table headers with sort indicators, hover effects, and results count displays with filtering status

Digital Tree Page Styles (admin.css)

Tree section and info display — Styled digital tree section with data table and visitor information display layout

Overlay Management Page Styles (admin.css)

Overlay cards grid and preview — Styled overlay cards in responsive grid layout with image previews, theme names, and action buttons

Add overlay modal form — Styled modal form for adding new overlay themes with file upload input and form validation

Question Management Page Styles (admin.css)

Question cards and metadata — Styled question display cards showing question text, type, status, and metadata in organized layout

Question options and actions — Styled question options display and action button groups for edit and delete operations

Add question modal form — Styled modal form for creating new questions with type selection, options management, and validation

User Management Page Styles (admin.css)

User section and edit modal — Styled user management table and modal form for editing user accounts with role selection and password fields

Audit Logs Page Styles (admin.css)

Audit section and empty states — Styled audit logs table section with empty state messaging when no logs are present

Loading spinner — Styled animated loading spinner with rotation animation for data loading states

Data Export Page Styles (admin.css)

Password protection panel — Styled locked state panel requiring password authentication before access to export features

Export cards and warning banners — Styled export option cards with file type icons and warning banners for sensitive data exports

Button Components (admin.css)

Primary, secondary, and warning buttons — Styled button variants with primary gradient background, secondary outline style, warning red theme, and hover/active states

View, edit, and delete action buttons — Styled icon-based action buttons with color coding (blue for view, yellow for edit, red for delete) and hover effects

Table Styles (admin.css)

Data tables and simple tables — Styled full-width data tables with striped rows, borders, and responsive behavior

Table header and cell styling — Styled table headers with bold text and bottom borders, and cells with padding and text alignment

Badge Components (admin.css)

Status and category badges — Styled inline badge components with rounded corners and color variants for success, warning, error, info, and custom states

Modal and Popup Styles (admin.css)

Photo, pledge, and QA popups — Styled large overlay popups for viewing photos, pledge content, and question-answer pairs with close buttons

Download and delete access modals — Styled password prompt modals for protected actions like downloads and deletions

Standard modal components — Styled base modal components with overlay background, centered content, headers, body sections, and action buttons

Utility Classes for Inline Style Replacement (admin.css)

Text and background color utilities — Defined utility classes for text colors and background colors matching semantic color scheme

Typography and spacing utilities — Defined utility classes for font sizes, weights, margins, and padding variations

Border and display utilities — Defined utility classes for borders, border radius, display properties, and flexbox layouts

Common modal component styles — Defined reusable utility classes for modal headers, bodies, footers, and action buttons

Animations (admin.css)

Fade, scale, and slide animations — Defined keyframe animations for fade-in/out, scale-up/down, and slide transitions

Loading spinner animation — Defined rotating animation keyframes for loading spinner indicator

Responsive Design (admin.css)

Tablet and mobile breakpoints — Defined media queries for tablet (max-width 968px) and mobile (max-width 768px) layouts

Layout adjustments for all components — Applied responsive layout changes including sidebar collapsing, grid column adjustments, reduced padding, and font size scaling

Theme Settings Page Styles (admin.css)

Theme settings container layout — Styled theme customization page with two-column layout for controls and preview

Theme sidebar and panel — Styled theme section sidebar with navigation list and color panel with grouped controls

Color pickers and preview — Styled color input controls with hex inputs and live preview area showing theme changes

Saved themes management — Styled saved themes section with theme cards, activation buttons, and management controls

Archive Deletion Controls Styling (admin.css)

Deletion controls container, header, and badge — Styled bulk deletion controls section with header, system admin badge, and warning indicators

Control group layout, labels, and counters — Styled control groups with labels, selected count displays, and organization layout

Date range inputs and select dropdowns — Styled date picker inputs and quick select dropdowns for deletion date ranges

Preview count and warning callouts — Styled preview count display and warning callout boxes for deletion confirmations

Checkbox column styling for bulk select — Styled checkbox column with select-all functionality and individual row selection

Warning and danger button variants — Styled warning-themed buttons for preview and danger-themed buttons for deletion with disabled states

Deletion confirmation modal layout and close button — Styled modal for confirming bulk deletions with close button and header

Modal warning block, backup checkbox, and typed confirmation input — Styled warning information block, backup reminder checkbox, and typed confirmation input field for safe deletion workflow

Mobile layout adjustments for deletion controls — Applied responsive adjustments for deletion controls on mobile including stacked layouts and adjusted spacing

Admin Panel HTML Structure

Login Page (admin.html)

Login page structure — Created login container with centered card layout, icon header, title and subtitle, login form with username and password inputs, submit button with handleLogin event handler, and test accounts display section listing

credentials for system admin, IT admin, and IT staff roles

Admin Dashboard (admin.html)

Admin dashboard container — Created main admin dashboard div with hidden initial display state containing header, sidebar, and all page content sections

Main content structure — Configured main content area wrapper for sidebar and page content sections

Page content container — Created page content container div to hold all individual page sections with display toggling

Dashboard Page (admin.html)

Dashboard page structure — Created dashboard page with active state, page header with title and description, stats grid containing five stat cards (total visitors, today's visitors, feedback submissions, users with email, recent submissions) each with icon, value placeholder, and label, activity section with refresh button and last updated timestamp, and two status sections for system status and data management

Status dynamic population — Created empty status section containers where JavaScript dynamically inserts status items for system status and data management categories

Feedback Data Page (admin.html)

Feedback data page structure — Created feedback page section with toolbar containing search and filter controls, advanced filter panel, results info bar, data table, and pagination controls

Global search input — Implemented search input field with placeholder and onchange event to filter feedback data by keyword

Advanced filter panel — Created collapsible filter panel with toggle button containing date range filter, email filter, visits filter, data retention filter, and results count display

Date range filter — Configured date range inputs with from and to date pickers and quick preset buttons for date selection

Email filter — Created email presence filter dropdown with options for all, with email, and without email

Visits filter — Created select dropdown for filtering by device type (all, desktop, mobile)

Data retention filter — Created select dropdown for filtering by retention period (all, 7 days, indefinite)

Results count display — Configured results counter showing filtered count out of total count of feedback records

Results info bar — Created info bar showing active filter count and result statistics with conditional display

Dynamic table population — Created table body container where JavaScript dynamically inserts feedback data rows

Digital Tree Page (admin.html)

Digital tree page structure — Created digital tree page section with page header, refresh button, and data table for displaying visitor information

Dynamic tree data — Created table body container where JavaScript populates tree visitor data rows

Overlay Management Page (admin.html)

Overlay management page structure — Created overlay page section with page header and overlay grid container for displaying theme cards

Dynamic overlay cards — Created empty overlay grid container where JavaScript inserts overlay theme cards with previews and actions

Question Management Page (admin.html)

Question management page structure — Created question page section with page header, add question button, and questions container for displaying question cards

Dynamic questions loading — Created empty questions container where JavaScript loads and displays question management cards

User Management Page (admin.html)

User management page structure — Created user page section with page header, add user button, and user table for displaying admin accounts

Dynamic user table — Created table body container where JavaScript populates user management rows

Archive Page (admin.html)

Archive page structure — Created archive page section with deletion controls, toolbar, filters, and data table

Deletion controls for system admin — Created deletion controls section visible only to system admin users with selected items deletion and date range deletion subsections

Selected items deletion — Configured bulk selection controls with select all toggle, selected count display, and delete selected button

Date range deletion — Configured date range deletion controls with date input, quick select dropdown, preview button, and bulk delete button

Audit Logs Page (admin.html)

Audit logs page structure — Created audit logs page with toolbar, advanced filters panel, results info bar, sortable table, and pagination controls

Action filter — Created action type filter dropdown with options for all actions, login, logout, delete, add, edit, download, and view

User filter — Created admin user filter dropdown with dynamic user population from audit data

Target type filter — Created target type filter dropdown with options for all targets, user, feedback, overlay, and question

IP address filter — Created IP address text input with placeholder for filtering by IP address

Audit results count — Configured results display showing filtered count out of total audit logs count

Audit results info bar — Created info bar displaying active filter count and filtered results with conditional visibility

Data Export Page (admin.html)

Data export page structure — Created data export page with password protection panel and export cards section

Password protection panel — Created locked state panel requiring system administrator verification with username display, password input, unlock form, warning badge, and informational text about sensitive data and session timeout

Export cards section — Created hidden export cards container revealed after successful authentication containing page header, warning bar with session info, and export grid with multiple export options

Full feedback export card — Created export card for downloading all feedback data (archived and not archived) in CSV format with decrypted emails

Feedback not archived export card — Created export card for downloading only active feedback records with not_archived status in CSV format

Feedback archived export card — Created export card for downloading only archived feedback records in CSV format

Photos export cards — Created export cards for downloading all photos, not archived photos, and archived photos in ZIP format

Audit log export card — Created export card for downloading complete audit trail in CSV format

Theme Settings Page (admin.html)

Theme settings page structure — Created theme settings page with page header and theme settings container with two-column layout

Theme sidebar section list — Created left sidebar with section list header and theme section list container, including saved themes item and dynamic population placeholder

Theme panel controls — Created right panel with panel header showing current section title, action buttons (reset to global, reset all to defaults, save changes), override toggle for non-global sections, and theme settings form

Override toggle section — Configured toggle switch with checkbox input and label for enabling or disabling global theme usage for specific sections

Theme settings form — Created form container where JavaScript dynamically loads color picker controls based on selected section

Save current theme section — Created save theme subsection with header, theme name input (max 100 characters), checkbox to set as active theme on save, and action buttons for saving or clearing the form

Saved themes list section — Created saved themes list subsection with header showing count out of maximum 6 themes and container for dynamically loaded theme cards

Dynamic theme controls — Created placeholder for dynamically populated theme controls based on section selection

Preview area — Created optional preview area for visualizing theme changes before saving

Admin Panel Client-Side Logic

Global Variables and State Management (admin.js)

Download operation tracking — Declared currentDownloadType variable to track which type of data export or download is currently active

Feedback deletion tracking — Declared currentDeleteFeedbackId variable to store the ID of feedback record pending deletion confirmation

Email viewing tracking — Declared currentEmailViewingId variable to store the ID of the email currently being viewed in popup

Audit pagination page — Declared currentAuditPage variable initialized to 1 for tracking current page in audit logs pagination

Audit items per page — Declared auditLogsPerPage variable set to 50 for controlling number of audit entries displayed per page

Audit sort field — Declared currentSortField variable initialized to 'created_at' for tracking which column audit logs are sorted by

Audit sort direction — Declared currentSortDirection variable initialized to 'desc' for tracking ascending or descending sort order

All audit logs storage — Declared allAuditLogs array to store complete set of audit log entries loaded from database

Filtered audit logs storage — Declared filteredAuditLogs array to store audit logs after applying search and filter criteria

All feedback data storage — Declared allFeedbackData array to store complete set of not_archived feedback records from database

Filtered feedback data storage — Declared filteredFeedbackData array to store feedback records after applying filters

Feedback pagination page — Declared feedbackCurrentPage variable initialized to 1 for tracking current page in feedback data view

Feedback items per page constant — Declared feedbackItemsPerPage constant set to 25 for controlling feedback records displayed per page

All archive data storage — Declared allArchiveData array to store complete set of archived feedback records from database

Filtered archive data storage — Declared filteredArchiveData array to store archive records after applying filters

Archive pagination page — Declared archiveCurrentPage variable initialized to 1 for tracking current page in archive data view

Archive items per page constant — Declared archiveItemsPerPage constant set to 25 for controlling archive records displayed per page

Decrypted email map — Declared decryptedEmailMap object to cache decrypted emails by feedback ID for current feedback page

Feedback decryption state flag — Declared isAnyEmailDecrypted boolean to track if any emails on current feedback page are decrypted

Encrypted feedback IDs array — Declared encryptedFeedbackIds array to store IDs of encrypted feedback entries on current page

Decrypted archive email map — Declared decryptedArchiveEmailMap object to cache decrypted emails by ID for current archive page

Archive decryption state flag — Declared isAnyArchiveEmailDecrypted boolean to track if any emails on current archive page are decrypted

Encrypted archive IDs array — Declared encryptedArchiveIds array to store IDs of encrypted archive entries on current page

Data export unlock status — Declared dataExportUnlocked boolean to track if user has successfully unlocked data export feature with password

Data export unlock timestamp — Declared dataExportUnlockTime variable to store when data export session was unlocked for session management

Question data map — Declared window.questionsDataMap as Map object to store question data globally to avoid JSON.stringify issues in onclick handlers

Notification display function — Implemented showNotification() to display user-facing messages with type-based styling (success, error, info, warning)

Authentication and Session Management (admin.js)

Admin login handler — Implemented handleLogin() to authenticate admin users via API, validate credentials, store session data in sessionStorage, load active theme, update UI based on role, and navigate to dashboard

Admin logout handler — Implemented handleLogout() to log logout action via audit API, clear sessionStorage, reset UI to login page, and clear form inputs

UI role-based update — Implemented updateUIForUser() to display username and role in header, show or hide administration section based on system_admin role, control download button visibility by role, and redirect non-admin users from restricted pages

Role name formatting — Implemented formatRoleName() to convert role codes (system_admin, IT_admin, IT_staff) to user-friendly display names

Audit Logs Management (admin.js)

Audit logs loading — Implemented loadAuditLogs() to fetch all audit log entries from API and populate both allAuditLogs and filteredAuditLogs arrays

User filter population — Implemented populateUserFilter() to extract unique usernames from audit logs and populate user filter dropdown with options

Audit logs filtering — Implemented filterAuditLogs() to apply multiple filter criteria including search text, action type, username, target entity, IP address, and date range to audit logs

Filtered logs sorting — Implemented sortFilteredLogs() to sort filtered audit logs by current sort field and direction using locale-aware comparison

Audit logs column sorting — Implemented sortAuditLogs() to handle column header clicks, toggle sort direction, update visual indicators, and trigger re-sorting of data

Audit logs page display — Implemented displayAuditLogsPage() to render current page of filtered audit logs with pagination controls

Audit table update — Implemented updateAuditTable() to generate HTML table rows for audit logs with badges, formatted timestamps, and escaped content

Audit badge type determination — Implemented getAuditBadgeType() to assign badge styles (success, info, warning, danger) based on action type keywords

Audit pagination update — Implemented updateAuditPagination() to calculate total pages, update page number display, and enable or disable navigation buttons

Previous audit page navigation — Implemented prevAuditPage() to decrement page number and refresh audit logs display

Next audit page navigation — Implemented nextAuditPage() to increment page number and refresh audit logs display

Advanced filters toggle — Implemented toggleAdvancedFilters() to show or hide advanced filter panel and update toggle button text

All filters reset — Implemented clearAllFilters() to reset all filter inputs to default values, clear applied filters, and refresh audit logs display

Date range preset setter — Implemented setDateRangePreset() to set start and end date inputs based on preset options (today, this week, this month)

Filter indicators update — Implemented updateFilterIndicators() to highlight filter controls that have active values applied

Results count update — Implemented updateResultsCounts() to display total and filtered count of audit log entries

Audit Excel download — Implemented downloadAuditExcel() to convert filtered audit logs to CSV format and trigger browser download

Dashboard Management (admin.js)

Dashboard data loading — Implemented loadDashboardData() to fetch dashboard statistics and recent activity from API and update UI elements

Dashboard statistics update — Implemented updateDashboardStats() to populate five stat cards with total visitors, today's visitors, feedback submissions, users with email, and recent submissions counts

Recent activity update — Implemented updateRecentActivity() to populate system status and data management sections with activity items and badges

Status item creation — Implemented createStatusItem() to generate HTML elements for individual status items with label, value, and colored badge

Last updated timestamp — Implemented updateLastUpdated() to display current time as last refresh timestamp in dashboard

Default statistics provider — Implemented getDefaultStats() to return fallback statistics object with zero values for offline or error scenarios

Dashboard refresh — Implemented refreshDashboard() to reload dashboard data by calling loadDashboardData function

Feedback Data Management (admin.js)

Feedback data loading — Implemented loadFeedbackData() to fetch all not_archived feedback records from API and populate allFeedbackData and filteredFeedbackData arrays

Feedback data filtering — Implemented filterFeedbackData() to apply multiple filter criteria including search text, date range, retention type, and device type to feedback records

Feedback counts update — Implemented updateFeedbackCounts() to display total and filtered count of feedback records

Feedback page rendering — Implemented renderFeedbackPage() to generate HTML table rows for current page of 25 feedback records with all action buttons and data fields

Feedback pagination controls update — Implemented updateFeedbackPaginationControls() to calculate total pages, update page display, and enable or disable navigation buttons

Previous feedback page navigation — Implemented prevFeedbackPage() to decrement page number and refresh feedback display

Next feedback page navigation — Implemented nextFeedbackPage() to increment page number and refresh feedback display

Feedback data refresh — Implemented refreshFeedbackData() to reload all feedback data and reset to first page

Archive Data Management (admin.js)

Archive data loading — Implemented loadArchiveData() to fetch all archived feedback records from API and populate allArchiveData and filteredArchiveData arrays

Archive data filtering — Implemented filterArchiveData() to apply search text, date range, retention type, and device type filters to archived records

Archive counts update — Implemented updateArchiveCounts() to display total and filtered count of archive records

Archive page rendering — Implemented renderArchivePage() to generate HTML table rows for current page of 25 archive records with selection checkboxes and action buttons

Archive pagination controls update — Implemented updateArchivePaginationControls() to calculate total pages, update page display, and enable or disable navigation buttons

Previous archive page navigation — Implemented prevArchivePage() to decrement page number and refresh archive display

Next archive page navigation — Implemented nextArchivePage() to increment page number and refresh archive display

Archive data refresh — Implemented refreshArchiveData() to reload all archive data and reset to first page

Filter Controls (admin.js)

Feedback advanced filters toggle — Implemented toggleFeedbackAdvancedFilters() to show or hide feedback advanced filter panel with smooth transition

Archive advanced filters toggle — Implemented toggleArchiveAdvancedFilters() to show or hide archive advanced filter panel with smooth transition

Feedback date range preset — Implemented setFeedbackDateRangePreset() to populate feedback date inputs based on preset selections (today, this week, this month)

Archive date range preset — Implemented setArchiveDateRangePreset() to populate archive date inputs based on preset selections (today, this week, this month)

All feedback filters clear — Implemented clearAllFeedbackFilters() to reset all feedback filter inputs to defaults and refresh display

All archive filters clear — Implemented clearAllArchiveFilters() to reset all archive filter inputs to defaults and refresh display

Encryption Management (admin.js)

Feedback encryption buttons update — Implemented updateEncryptionButtons() to show decrypt button when encrypted emails exist and re-encrypt button when any emails are decrypted on current feedback page

Archive encryption buttons update — Implemented updateArchiveEncryptionButtons() to show decrypt button when encrypted emails exist and re-encrypt button when any emails are decrypted on current archive page

All feedback emails decryption — Implemented decryptAllEmails() to prompt for password, decrypt all encrypted emails on current feedback page, cache results, and update display

All archive emails decryption — Implemented decryptAllArchiveEmails() to prompt for password, decrypt all encrypted emails on current archive page, cache results, and update display

All feedback emails re-encryption — Implemented reEncryptAllEmails() to clear decryption cache, reset decryption flags, refresh feedback display to show encrypted state again

All archive emails re-encryption — Implemented reEncryptAllArchiveEmails() to clear archive decryption cache, reset flags, refresh archive display to show encrypted state again

Pledge and Content Viewing (admin.js)

Pledge viewing — Implemented viewPledge() to display pledge text content in centered popup modal with close functionality

Pledge popup close — Implemented closePledgePopup() to hide pledge display modal

Question answers viewing — Implemented viewQuestionAnswers() to fetch question and answer data from API and display in formatted popup interface

Question answers popup creation — Implemented createQuestionAnswersPopup() to generate HTML structure for displaying question-answer pairs with proper formatting

Answer formatting — Implemented formatAnswer() to format answer display based on question type (rating shows stars, text shows content, or combined view)

Question answers popup close — Implemented closeQAPopup() to hide question answers display modal

Photo Management (admin.js)

Raw photo viewing — Implemented viewRawPhoto() to fetch and display original unprocessed photo in popup without requiring password authentication

Processed photo viewing — Implemented viewProcessedPhoto() to fetch and display overlay-applied photo in popup without requiring password authentication

Raw photo popup display — Implemented showRawPhotoPopup() to create and show modal containing raw photo image with download and close buttons

Processed photo popup display — Implemented showProcessedPhotoPopup() to create and show modal containing processed photo image with download and close buttons

Photo popup creation — Implemented createPhotoPopup() to generate reusable HTML structure for photo viewing modals with image display and action buttons

Photo error handling — Implemented handlePhotoError() to display error message when photo fails to load and provide close functionality

Photo download — Implemented downloadPhoto() to trigger browser download of photo file with appropriate filename

Photo popup close — Implemented closePhotoPopup() to hide photo viewing modal and remove from DOM

Email Management (admin.js)

Password prompt display — Implemented showPasswordPrompt() to create custom modal with hidden password input field for secure entry

Email popup display — Implemented showEmailPopup() to create and show modal containing decrypted email address with copy functionality

Email popup close — Implemented closeEmailPopup() to hide email display modal and remove from DOM

Overlay Management (admin.js)

Overlay data loading — Implemented loadOverlayData() to fetch overlay themes from API and update overlay grid display

Overlay message display — Implemented showOverlayMessage() to display status messages for overlay operations with type-based styling

Overlays table creation — Implemented createOverlaysTable() as placeholder function for future table-based overlay display

Overlay table update — Implemented updateOverlayTable() to generate grid of overlay cards with preview images, theme names, and action buttons

Overlay viewing — Implemented viewOverlay() to display overlay image in full-size popup modal with download functionality

Overlay preview close — Implemented closeOverlayPreview() to hide overlay preview modal

Overlay image error handling — Implemented handleOverlayImageError() to display error message when overlay image fails to load

Overlay download — Implemented downloadOverlay() to trigger browser download of overlay image file

Add overlay modal display — Implemented showAddOverlayModal() to show overlay creation form for system admin users only

Add overlay modal close — Implemented closeAddOverlayModal() to hide overlay creation form and reset inputs

Overlay creation handler — Implemented handleAddOverlay() to process new overlay submission with file upload, validation, and API communication

Overlay deletion request — Implemented deleteOverlay() to prompt for confirmation and initiate overlay deletion for system admin users only

Overlay deletion execution — Implemented performOverlayDeletion() to execute confirmed overlay deletion via API and refresh display

User Management (admin.js)

User management data loading — Implemented loadUserManagementData() to fetch admin users list from API and populate user management table

User management table update — Implemented updateUserManagementTable() to generate table rows for admin users with role display and action buttons for system admins

User addition modal display — Implemented addUser() to show user creation form for system admin users only

User addition handler — Implemented handleAddUser() to validate inputs, check password match, create new admin user via API, and refresh user list

Add user modal close — Implemented closeAddUserModal() to hide user creation form and reset all input fields

User editing modal display — Implemented editUser() to populate and show user edit form with existing user data for system admin users only

User editing handler — Implemented handleEditUser() to validate changes, update user information via API, and refresh user list

Edit user modal close — Implemented closeEditUserModal() to hide user edit form and reset inputs

User deletion request — Implemented deleteUser() to prompt for confirmation and initiate user deletion for system admin users only

User deletion execution — Implemented performUserDeletion() to execute confirmed user deletion via API and refresh user list

Question Management (admin.js)

Question management data loading — Implemented loadQuestionManagementData() to fetch questions from API, populate window.questionsDataMap, and update question table

Question management table update — Implemented updateQuestionManagementTable() to generate table rows for questions with type, status, and action buttons for system admins

Question type formatting — Implemented formatQuestionType() to convert question type codes to user-friendly display names (Rating, Text Input, Combined)

Add question modal display — Implemented showAddQuestionModal() to show question creation form for system admin users only

Add question modal close — Implemented closeAddQuestionModal() to hide question creation form and reset inputs

Options field toggle — Implemented toggleOptionsField() to show or hide options input section based on selected question type

Option field addition — Implemented addOptionField() to dynamically add new option input field to form

Option field removal — Implemented removeOptionField() to remove specific option input field from form

Question addition handler — Implemented handleAddQuestion() to validate inputs, create new question via API with proper type handling, and refresh question list

Question retrieval helper — Implemented editQuestionById() to retrieve question data from Map and call editQuestion function

Question editing modal display — Implemented editQuestion() to populate and show question edit form with existing question data for system admin users only

Question editing handler — Implemented handleEditQuestion() to safely update questions with validation, handle type changes, update via API, and refresh question list

Edit question modal close — Implemented closeEditQuestionModal() to hide question edit form and reset inputs

Question deletion — Implemented deleteQuestion() to prompt for confirmation, delete question via API, and refresh question list for system admin users only

Data Export Management (admin.js)

Data export page initialization — Implemented initDataExportPage() to check if user is system admin and show locked state or unlock interface based on role

Data export unlock — Implemented unlockDataExport() to verify admin password via API, set session unlock flag and timestamp, and reveal export controls

Export session info update — Implemented updateExportSessionInfo() to display unlocked timestamp and session status information

Export file download — Implemented downloadExport() to validate export type selection, generate appropriate file format (Excel, CSV, or Zip), and trigger download

Feedback Deletion (admin.js)

Feedback deletion request — Implemented deleteFeedback() to prompt for confirmation, show password modal, and initiate feedback deletion workflow

Archive feedback deletion request — Implemented deleteArchiveFeedback() to prompt for confirmation, show password modal, and initiate archive feedback deletion workflow

Delete password modal display — Implemented showDeletePasswordModal() to create and show password verification modal for deletion operations

Delete access verification — Implemented verifyDeleteAccess() to validate entered password against API before allowing deletion to proceed

Feedback deletion execution — Implemented performFeedbackDeletion() to execute confirmed feedback deletion via API, log action, and refresh data display

Delete modal close — Implemented closeDeleteModal() to hide deletion password modal and reset state

Navigation and Page Management (admin.js)

Page navigation — Implemented showPage() to handle navigation between admin pages with role-based access control, hide all pages, show selected page, update sidebar active state, and load page-specific data

Archive page initialization — Implemented initializeArchivePage() as placeholder function for future archive page initialization logic

Digital Tree Management (admin.js)

Digital tree data loading — Implemented loadDigitalTreeData() to fetch visitor data from tree API and update tree visitor table

Digital tree table update — Implemented updateDigitalTreeTable() to generate table rows for tree visitors with names, messages, visit dates, and device information

Tree data refresh — Implemented refreshTreeData() to reload digital tree visitor data from API

Utility Functions (admin.js)

HTML escaping — Implemented escapeHtml() to sanitize user input by replacing special characters with HTML entities to prevent XSS attacks

Theme Settings Helper Functions (admin.js)

Confirmation dialog display — Implemented showConfirmDialog() to create and show custom confirmation modal with message and callback handling

Theme Configuration (admin.js)

Default theme settings provider — Implemented getDefaultThemeSettings() to return comprehensive default color configuration for all admin pages and components

Theme settings initialization — Implemented initThemeSettings() to load saved theme settings or defaults and populate theme customization interface

Theme settings loading — Implemented loadThemeSettings() to retrieve theme configuration from localStorage, migrate old formats, and return settings object

Old colors migration — Implemented migrateOldColors() to convert legacy color format to new structured theme configuration

Custom default colors provider — Implemented getCustomDefaultColors() to return fallback color values for theme system

Individual page settings save — Implemented saveIndividualPageSettings() to store page-specific color overrides in theme configuration

Individual page settings load — Implemented loadIndividualPageSettings() to retrieve page-specific color overrides from theme configuration

Theme settings save — Implemented saveThemeSettings() to persist theme configuration to localStorage and trigger save confirmation message

Save message display — Implemented showSaveMessage() to show temporary success notification after theme settings are saved

Theme Management Functions (admin.js)

Section list rendering — Implemented renderSectionList() to generate list of all available theme sections for selection interface

Section selection — Implemented selectSection() to handle section selection, update active state, and load section-specific color controls

Section title update — Implemented updateSectionTitle() to display current section name and override status in header

Theme controls rendering — Implemented renderThemeControls() to generate color picker controls organized by groups for selected section

Color group creation — Implemented createColorGroup() to generate grouped color controls with collapsible header and contained color pickers

Color control creation — Implemented createColorControl() to generate individual color picker with label, hex input, and value preview

Color value update — Implemented updateColor() to handle color picker changes, update hex input, save to settings, and refresh preview

Color hex input update — Implemented updateColorFromHex() to handle hex input changes, validate format, update color picker, save to settings, and refresh preview

Global theme override toggle — Implemented toggleGlobalThemeOverride() to enable or disable page-specific color overrides and switch between global and custom colors

Section reset to global — Implemented resetSectionToGlobal() to remove page-specific overrides and restore global theme colors for section

All settings reset to defaults — Implemented resetAllToDefaults() to prompt for confirmation, clear all custom settings, restore default theme configuration, and refresh interface

Theme preview update — Implemented updatePreview() to apply current theme colors to live preview area showing affected UI elements

Theme settings application — Implemented applyThemeSettings() to read theme configuration and apply colors to all admin page elements via CSS custom properties

Saved Themes Management (admin.js)

Saved themes cache — Declared savedThemesCache variable to store loaded saved themes data for quick access

Saved themes loading — Implemented loadSavedThemes() to fetch user's saved themes from database API and populate cache

Saved themes UI update — Implemented updateSavedThemesUI() to generate theme cards for all saved themes or show empty state message

Saved theme card creation — Implemented createSavedThemeCard() to generate HTML card element with theme preview colors, name, timestamps, and action buttons

Preview colors extraction — Implemented extractPreviewColors() to select representative colors from theme data for card preview display

Current theme data capture — Implemented captureCurrentThemeData() to serialize current theme configuration including all colors and section overrides

Current theme save — Implemented saveCurrentTheme() to validate theme name, capture configuration, save to database via API, and refresh saved themes list

Save theme form clear — Implemented clearSaveThemeForm() to reset theme name input and close save theme modal

Theme data application — Implemented applyThemeData() to deserialize and apply saved theme configuration to UI, update settings, and refresh preview

Saved theme activation — Implemented activateSavedTheme() to set theme as user's active theme in database and apply immediately to interface

Saved theme rename — Implemented renameSavedTheme() to show rename modal, validate new name, update via API, and refresh themes list

Rename modal creation — Implemented createRenameModal() to generate HTML modal interface for theme renaming with input validation

Rename operation execution — Implemented performRename() to validate input, call API to rename theme, update cache, and refresh display

Rename modal close — Implemented closeRenameModal() to hide rename modal and remove from DOM

Saved theme deletion — Implemented deleteSavedTheme() to prompt for confirmation, delete theme from database via API, and refresh themes list

Saved themes list refresh — Implemented refreshSavedThemes() to reload saved themes from database and update UI display

Active theme login load — Implemented loadActiveThemeOnLogin() to fetch user's active theme on login and apply configuration automatically

Default theme application — Implemented applyDefaultTheme() to apply standard default colors when no custom theme is active

Saved themes section initialization — Implemented initSavedThemesSection() to load saved themes and populate interface on page load

Archive Deletion Functions (admin.js)

Deletion controls initialization — Implemented initializeDeletionControls() to show or hide bulk deletion controls based on user role (system admin only)

Select all archive toggle — Implemented toggleSelectAllArchive() to check or uncheck all archive record selection checkboxes and update delete button state

Archive selection count update — Implemented updateArchiveSelectionCount() to count selected checkboxes, display count, and enable or disable bulk delete button

Quick delete date setter — Implemented setQuickDeleteDate() to populate bulk delete date input with preset values (7 days, 30 days, 90 days ago)

Bulk delete preview — Implemented previewBulkDelete() to query API for count of records before specified date and display confirmation with count

Selected archive deletion — Implemented deleteSelectedArchive() to collect selected record IDs, prompt for confirmation, delete via API, and refresh archive data

Bulk delete by date — Implemented bulkDeleteByDate() to delete all archive records before specified date via API after confirmation

Deletion confirmation modal display — Implemented showDeletionConfirmationModal() to create and show custom confirmation modal for bulk deletion operations

Archive data refresh — Implemented refreshArchiveData() as duplicate function to reload archive data after deletion operations

Admin API Routes and Management Logic

Audit Logging Functions (adminRoutes.js)

Admin action audit logging — Implemented logAudit() function to record important admin actions (login, logout, deletions, additions, edits, downloads, email views, theme operations) to audit_logs table with action type, admin username, target type and ID, IP address from request headers, and user agent, using fire-and-forget database insert

File Upload Configuration (adminRoutes.js)

Multer storage configuration — Configured multer.diskStorage to handle overlay file uploads with destination routing based on fieldname (desktop_file to DesktopOverlay folder, mobile_file to MobileOverlay folder), automatic directory creation, and filename generation using theme ID pattern

Multer upload handler — Configured multer instance with storage settings, PNG-only file filter validation, and 10MB file size limit for overlay image uploads

Authentication Routes (adminRoutes.js)

Admin login — Implemented POST /login endpoint to authenticate admin users using auth.loginUser callback, store user session data (id, username, role) in req.session, log successful login to audit trail, and return user object with success message

Admin logout audit — Implemented POST /logout-audit endpoint to log admin logout action to audit trail, destroy user session, and return success confirmation

Dashboard Routes (adminRoutes.js)

Dashboard statistics retrieval — Implemented GET /dashboard endpoint to calculate current month date range, fetch five statistics (total visitors, today's visitors, feedback submissions, users with email, recent submissions from last 7 days) using separate database queries with date filtering, compile stats object, and return dashboard data with recent activity sections

Database connection test — Implemented GET /test-db endpoint to verify database connectivity and return table count information

Feedback Management Routes (adminRoutes.js)

All feedback retrieval — Implemented GET /feedback endpoint to fetch all active not_archived feedback records with user data, encrypted emails, visits, pledge, retention, photo paths, email status, archive status, joined with question answers, and ordered by creation date descending

Feedback update — Implemented PUT /feedback/:id endpoint to update feedback entry allowing admin to modify pledge comment and admin notes fields

Feedback deletion — Implemented DELETE /feedback/:id endpoint to permanently delete feedback with cascade deletion of question answers, photo file cleanup from filesystem (both raw and processed photos), and audit logging

Feedback questions and answers retrieval — Implemented GET /feedback/:id/questions endpoint to fetch all questions with their corresponding answers for specific feedback entry with joined query

Archive Management Routes (adminRoutes.js)

Archived feedback retrieval — Implemented GET /archive endpoint to fetch feedback records with archive_status of 'archived' (automatically set for records older than 3 months) including all user data and question answers

Manual archive status update — Implemented POST /archive/update-status endpoint to manually trigger archive status update process for records older than 90 days

Archive statistics — Implemented GET /archive/stats endpoint to return count of archived records for monitoring purposes

Bulk archive email decryption — Implemented POST /bulk-decrypt-archive endpoint to verify admin password and decrypt multiple archived email addresses in batch operation

Archive photos download — Implemented POST /download-archive-photos endpoint to create ZIP archive of selected archived feedback photos and provide download link

Download file serving — Implemented GET /download-file/:filename endpoint to serve generated export files from temporary directory with proper content disposition headers

Archive deletion preview — Implemented POST /archive/preview-deletion endpoint for system admin only to query and return count of records matching deletion criteria before executing bulk delete

Archive selected deletion — Implemented POST /archive/delete-selected endpoint for system admin only to permanently delete selected archived feedback records by ID array with photo cleanup, answer cascade deletion, and audit logging

Archive date range deletion — Implemented POST /archive/delete-by-date endpoint for system admin only to permanently delete all archived feedback before specified date with cascade operations and audit logging

Photo Access and Email Decryption Routes (adminRoutes.js)

Photo access verification — Implemented POST /verify-photo-access endpoint to verify system admin password before allowing access to view photos

Email decryption — Implemented POST /decrypt-email endpoint to verify system admin credentials and decrypt single encrypted email address using auth.decryptEmail function

Admin User Management Routes (adminRoutes.js)

Admin users retrieval — Implemented GET /users endpoint to fetch all admin users with role information from admin_users table

Admin user deletion — Implemented DELETE /users/:id endpoint to perform soft delete of admin user by setting is_active flag to 0 with audit logging

Admin user creation — Implemented POST /users endpoint to create new admin user with username, password hashing using bcrypt, role assignment, and audit logging for system admin only

Admin user update — Implemented PUT /users/:id endpoint to update admin user details including username, role, and optionally password with validation and audit logging for system admin only

Data Export Management Routes (adminRoutes.js)

Data export unlock — Implemented POST /data-export/unlock endpoint to verify system admin password and grant temporary access to data export features with session management

Export and Import Routes (adminRoutes.js)

Feedback Excel download — Implemented GET /download-excel endpoint to export all feedback data as CSV file with decrypted emails, user information, question answers, and metadata for system admin only with audit logging

Archive Excel download — Implemented GET /download-archive-excel endpoint to export archived feedback as CSV with decryption for system admin only with audit logging

Photos ZIP download — Implemented GET /download-photos endpoint to create ZIP archive of all uploaded photos (raw and processed) and trigger download for system admin only with audit logging

Overlay Management Routes (adminRoutes.js)

Overlays retrieval — Implemented GET /overlays endpoint to fetch all overlay themes with display order, theme names, file paths for desktop and mobile versions from overlays table

Overlay creation — Implemented POST /overlays endpoint for system admin only to add new overlay theme with file uploads for desktop and mobile PNG images, database record creation, and audit logging

Overlay deletion — Implemented DELETE /overlays/:id endpoint to remove overlay record from database, delete associated desktop and mobile image files from filesystem, and log action to audit trail

Question Management Routes (adminRoutes.js)

Questions retrieval — Implemented GET /questions endpoint to fetch all active questions with their multiple choice options ordered by display order

Question creation — Implemented POST /questions endpoint to create new question with question text, type (rating/text/combined), required flag, display order, and associated multiple choice options insertion

Question deletion — Implemented DELETE /questions/:id endpoint to perform soft delete (set is_active to 0) if question has existing answers, or hard delete if no answers exist, with cascade deletion of question options

Question update — Implemented PUT /questions/:id endpoint to safely update question text, type, required status, and display order without breaking existing answer references

Audit Logs Routes (adminRoutes.js)

Audit logs retrieval — Implemented GET /audit-logs endpoint to fetch audit log entries with pagination support, filtering options, and ordered by creation date descending

Helper Functions (adminRoutes.js)

User photo deletion — Implemented deleteUserPhotos() function to delete both raw and processed photo files from filesystem based on photo paths stored in feedback record

Overlay file deletion — Implemented deleteOverlayFiles() function to remove desktop and mobile overlay PNG files from assets directory

Directory photo check — Implemented checkDirectoryForPhotos() function to verify if directory contains image files with supported extensions (jpg, jpeg, png, gif)

Uploads ZIP creation — Implemented createUploadsZip() function to archive entire uploads directory into ZIP file for bulk download

CSV conversion — Implemented convertToCSV() function to transform JavaScript data array into comma-separated values format with proper header row and quoted fields

Saved Themes Routes (adminRoutes.js)

Saved themes retrieval — Implemented GET /saved-themes endpoint to fetch all saved theme presets for currently logged-in user with theme data and active status

Theme save — Implemented POST /saved-themes endpoint to create new theme preset for current user with theme name, configuration data, optional auto-activation, and maximum 6 themes limit enforcement with audit logging

Theme activation — Implemented PUT /saved-themes/:id/activate endpoint to set specific saved theme as active theme for current user by updating is_active flags with audit logging

All themes deactivation — Implemented POST /saved-themes/deactivate-all endpoint to set all user themes to inactive state resetting to default theme

Theme update — Implemented PUT /saved-themes/:id endpoint to rename saved theme with validation and audit logging

Theme deletion — Implemented DELETE /saved-themes/:id endpoint to remove saved theme from database with audit logging

Active theme retrieval — Implemented GET /saved-themes/active endpoint to fetch currently active theme configuration for logged-in user or return null if using default theme

---

SUPPORTING MODULES

Authentication and Security Functions

Configuration (auth.js)

bcrypt salt rounds configuration — Defined SALT_ROUNDS constant set to 12 for controlling bcrypt hashing complexity balancing security and performance

Email encryption key — Configured ENCRYPTION_KEY from environment variable with fallback to randomly generated 32-byte hex string for securing email data

AES-256-GCM algorithm constant — Defined ENCRYPTION_ALGORITHM as 'aes-256-gcm' for authenticated encryption of email addresses

Password Hashing Functions (auth.js)

Asynchronous password hashing — Implemented hashPassword() async function to hash passwords using bcrypt with configured salt rounds and return hashed password or throw error

Synchronous password hashing — Implemented hashPasswordSync() function to hash passwords synchronously using bcrypt for cases requiring blocking operation

Password verification — Implemented verifyPassword() async function to compare plain text password against bcrypt hash and return boolean match result

Email Encryption Functions (auth.js)

Email encryption — Implemented encryptEmail() function to encrypt email addresses using AES-256-GCM with random initialization vector, authentication tag, and hex-encoded output format (iv:authTag:encryptedData)

Email decryption — Implemented decryptEmail() function to decrypt email addresses by parsing encrypted format, extracting IV and auth tag, using AES-256-GCM decipher with authentication verification, and returning plain text email

Authentication Middleware (auth.js)

Authentication requirement — Implemented requireAuth() middleware function to check if user session exists and has user object, allowing request to proceed or returning 401 unauthorized error

Admin role requirement — Implemented requireAdmin() middleware function to verify user session has system_admin role, allowing request to proceed or returning 403 forbidden error

User Authentication Functions (auth.js)

User login with bcrypt — Implemented loginUser() function to validate username and password inputs, query admin_users table for active user, verify password using bcrypt comparison, update last_login timestamp on success, and return user object without password via callback

Admin User Management Functions (auth.js)

Admin users retrieval — Implemented getAdminUsers() function to fetch all admin users from database with id, username, role, last_login, created_at, and is_active fields ordered by id

Admin user creation — Implemented addAdminUser() async function to validate inputs, hash password using bcrypt, insert new admin user record with username, hashed password, and role, handle unique constraint violations, and return new user object via callback

Admin user update — Implemented updateAdminUser() async function to validate inputs, optionally hash new password if provided, update username and role fields, handle constraint violations, verify user exists by checking changes count, and return update result via callback

Admin user deletion — Implemented deleteAdminUser() function to permanently delete admin user record from database by ID, verify deletion occurred by checking changes count, and return deletion result via callback

Data Export API Routes and Functions

Imports and Configuration (dataExportRoutes.js)

Express framework import — Imported express module for creating router instance and handling HTTP routes

Router instance creation — Created express.Router instance for organizing data export route handlers

Authentication module import — Imported auth module for password verification and email decryption functions

Database module import — Imported db module for executing database queries to retrieve export data

File system module import — Imported fs module for checking file existence and reading photo files

Path module import — Imported path module for constructing file paths to uploads directories

Archiver library import — Imported archiver module for creating ZIP archives of photo collections

Middleware Functions (dataExportRoutes.js)

Data export unlock verification — Implemented requireDataExportUnlocked() middleware to verify user session exists, check system_admin role, validate data export unlock flag in session, enforce 30-minute timeout from unlock time, and reject expired or missing unlock sessions

Session Unlock Routes (dataExportRoutes.js)

Data export unlock — Implemented POST /unlock endpoint to verify admin username and password using auth.loginUser, check system_admin role requirement, set session flags (data_export_unlocked and data_export_unlock_time), explicitly save session before responding, log unlock action to audit trail, and return 30-minute expiration time

Feedback Export Routes (dataExportRoutes.js)

Full feedback export — Implemented GET /feedback/full endpoint to retrieve all feedback records without archive filter, decrypt emails using decryptEmailsInFeedback, generate CSV file with generateFeedbackExcel, set CSV content headers, and log export action to audit trail

Not archived feedback export — Implemented GET /feedback/not-archived endpoint to retrieve feedback with archive_status of 'not_archived', decrypt emails, generate CSV export, and log action

Archived feedback export — Implemented GET /feedback/archived endpoint to retrieve feedback with archive_status of 'archived', decrypt emails, generate CSV export, and log action

Photo Export Routes (dataExportRoutes.js)

All photos export — Implemented GET /photos/all endpoint to export all photos without archive filter using exportPhotos function and log action to audit trail

Not archived photos export — Implemented GET /photos/not-archived endpoint to export photos for not_archived feedback only as ZIP archive with logging

Archived photos export — Implemented GET /photos/archived endpoint to export photos for archived feedback only as ZIP archive with logging

Audit Log Export Routes (dataExportRoutes.js)

Audit log export — Implemented GET /audit-log endpoint to query all audit_logs table records ordered by created_at descending, convert to CSV format, set download headers, and log export action

Helper Functions - Email Decryption (dataExportRoutes.js)

Bulk email decryption — Implemented decryptEmailsInFeedback() function to iterate through feedback array, decrypt each encrypted email using auth.decryptEmail, replace encrypted field with decrypted value, handle decryption failures with error message placeholder, and handle empty or missing emails

Helper Functions - Data Retrieval (dataExportRoutes.js)

Feedback data retrieval — Implemented getFeedbackData() function to query feedback and users tables with optional archive_status filter, fetch all questions ordered by display order, fetch all feedback answers joined with questions,

organize answers by feedback_id for efficient lookup, and return promise with feedback rows, questions, and answers organized by feedback

Helper Functions - Excel Generation (dataExportRoutes.js)

Feedback CSV generation — Implemented generateFeedbackExcel() async function to build headers array with feedback fields and dynamic question columns, iterate through feedback records creating rows with all fields and corresponding answers matched by question ID, use already-decrypted emails from feedback array, and convert to CSV format using convertToCSV

Helper Functions - Photo Export (dataExportRoutes.js)

Photos ZIP archive creation — Implemented exportPhotos() async function to query feedback table for photo paths with optional archive filter, count raw and processed photos, create readme.txt with export metadata and file counts, initialize archiver with maximum compression, stream ZIP to response with appropriate headers, add photos from uploads/photos and uploads/processed directories to archive structure, and finalize archive

Utility Functions (dataExportRoutes.js)

CSV conversion — Implemented convertToCSV() function to handle both array of arrays format (headers plus data rows) and array of objects format, escape double quotes in cell values, wrap all cells in quotes, and join with commas and newlines

Timestamp generation — Implemented getTimestamp() function to format current date as ISO string, replace colons and periods with hyphens, and extract date portion for filename timestamps

Audit logging — Implemented logAudit() function to extract IP address and user agent from request headers, insert audit log record with action, admin username, target type and ID, IP address, and user agent into audit_logs table with fire-and-forget pattern

Module Exports (dataExportRoutes.js)

Router export — Exported configured router instance with all data export routes and middleware for mounting in main application

Data Retention and Cleanup Automation

Core Cleanup Functions (dataRetentionCleanup.js)

Data expiration calculation — Implemented isExpired() function to determine if feedback data has expired by checking retention period (returning false for indefinite), calculating days difference between Singapore timezone current time (UTC+8) and creation date for 7-day retention, and returning true if 7 or more days have passed

Photo file deletion — Implemented deletePhotoFile() function to construct full filesystem path from relative photo path, check file existence, delete file using fs.unlinkSync, log deletion success or error, and handle missing files gracefully

Expired data cleanup — Implemented cleanupExpiredData() function to query all active feedback with 7-day retention joined with users table, iterate through results checking expiration with isExpired, delete photo files from filesystem for expired entries, check if user has any indefinite retention feedback before clearing email, clear encrypted email from users table only if no indefinite feedback exists, update feedback table to set photo_path and processed_photo_path to NULL, and log detailed cleanup summary with counts

Initialization and Scheduling (dataRetentionCleanup.js)

Cleanup system initialization — Implemented initializeCleanup() function to schedule initial cleanup 5 seconds after server startup, set recurring cleanup interval every 6 hours using setInterval, and log cleanup schedule configuration

Manual cleanup trigger — Implemented runManualCleanup() function to allow admin-triggered cleanup execution outside scheduled intervals with audit logging

Module Exports (dataRetentionCleanup.js)

Function exports — Exported initializeCleanup, runManualCleanup, and cleanupExpiredData functions for use in main server application and admin routes

Database Connection and Query Interface

Database Configuration (db.js)

MySQL driver import — Imported mysql2 module for MySQL database connectivity with connection pooling support

Path utility import — Imported path module for file path operations

MySQL connection pool creation — Created mysql2 connection pool with environment-based configuration (host, user, password, database name) with fallback to default development credentials (kiosk_user/kiosk123/dp_kiosk_db), connection pooling settings (10 concurrent connections, unlimited queue), and keep-alive enabled for persistent connections

Database Connection Test (db.js)

Startup connection verification — Tested database connection on server startup using pool.getConnection, logged success message with database name, released connection back to pool, or exited process with error message and troubleshooting checklist if connection failed

SQLite-Compatible Wrapper Functions (db.js)

Single row query execution — Implemented get() function to execute SQL query with parameters, return first row or null mimicking SQLite db.get behavior, handle missing callback gracefully, and invoke callback with error or single row result

All rows query execution — Implemented all() function to execute SQL query with parameters, return all rows as array mimicking SQLite db.all behavior, handle missing callback gracefully, and invoke callback with error or results array

Insert/update/delete execution — Implemented run() function to execute INSERT/UPDATE/DELETE queries with parameters, mimic SQLite db.run behavior by creating context object with lastID (insertId) and changes (affectedRows) properties, call callback with context bound to this for SQLite compatibility, and handle missing callback by executing query without response

Transaction Functions (db.js)

Transaction begin — Implemented beginTransaction() function to execute START TRANSACTION SQL command and invoke callback

Transaction commit — Implemented commit() function to execute COMMIT SQL command and invoke callback

Transaction rollback — Implemented rollback() function to execute ROLLBACK SQL command and invoke callback

Module Exports (db.js)

Database exports — Exported pool for raw MySQL access, get/all/run functions for SQLite-compatible query interface, query function as direct pool.query wrapper, and transaction helper functions (beginTransaction, commit, rollback) for transactional operations

---

Extra Modules

Database Security Migration Script

Imports and Configuration (AuthLayer_Reconstitution.js)

MySQL2 promise library import — Imported mysql2/promise module for async database operations with promise-based interface

Bcrypt library import — Imported bcrypt module for secure password hashing using bcrypt algorithm

Crypto module import — Imported Node.js crypto module for email encryption operations

Database connection configuration — Defined DB_CONFIG object with localhost host, kiosk_user username, kiosk123 password, and dp_kiosk_db database name for MySQL connection

Encryption key configuration — Defined ENCRYPTION_KEY constant from environment variable with fallback to placeholder value for AES-256 email encryption

Helper Functions (AuthLayer_Reconstitution.js)

Password hashing — Implemented hashPassword() async function to hash plain text passwords using bcrypt with 12 salt rounds and return hashed password

Email encryption — Implemented encryptEmail() function to encrypt email addresses using AES-256-GCM algorithm with random 16-byte initialization vector, authentication tag, and return formatted string (iv:authTag:encryptedData)

Password Migration (AuthLayer_Reconstitution.js)

Admin password migration — Implemented migratePasswords() async function to query all admin users from admin_users table, check if passwords are already bcrypt hashed by detecting $2b$ or $2a$ prefix, hash plain text passwords using bcrypt with 12 rounds, update database with hashed passwords, track migration statistics (already hashed, newly migrated, errors), and log summary with important notice about original password usage

Email Migration (AuthLayer_Reconstitution.js)

User email migration — Implemented migrateEmails() async function to check for email and email_encrypted columns existence in users table using information schema query, query all users with non-null emails, detect already encrypted emails by checking for colon separator in encrypted format, determine which email field to encrypt based on column availability, encrypt plain text emails using AES-256-GCM, update database setting email to NULL and email_encrypted to encrypted value, track migration statistics (already encrypted, newly encrypted, no email, errors), and log detailed summary

Main Execution (AuthLayer_Reconstitution.js)

Complete migration orchestration — Implemented runMigration() async function to create MySQL connection using DB_CONFIG, execute migratePasswords for admin user password hashing, execute migrateEmails for user email encryption, log success message with next steps for testing, handle specific error codes (access denied, connection refused, bad database) with targeted error messages, close database connection in finally block, and exit process with code 1 on failure

Script entry point — Called runMigration() function to execute complete migration process when script is run directly

Database Diagnostic Utility

Imports and Configuration (check_database.js)

MySQL library import — Imported mysql2 module for database connectivity and query execution

Database connection configuration — Defined DB_CONFIG object with localhost host, kiosk_user username, kiosk123 password, and dp_kiosk_db database name

MySQL connection instance creation — Created mysql2 connection instance using DB_CONFIG credentials for diagnostic queries

Database Connection and Verification (check_database.js)

Database connection establishment — Connected to MySQL database using configured credentials, logged successful connection with database name, and exited with error message if connection failed

Admin users retrieval — Executed query to select all admin users with id, username, full_name, role, is_active status, password hash preview (first 20 characters), hash length, created_at, and last_login timestamps ordered by id

User information display — Iterated through query results displaying each user's complete details including ID, username, full name, role, active status, password hash preview and length, creation date, and last login timestamp

Bcrypt hash validation — Checked password hash format by detecting $2b$, $2a$, or $2y$ prefix to validate proper bcrypt hashing and warned if hash appears to be plain text

Inactive account warning — Logged warning message for any users with is_active flag set to 0 indicating account cannot login

Authentication query test — Executed test query matching auth.js login logic for systemadmin user with is_active = 1 filter, displayed query results or diagnostic messages for no results with possible reasons (case sensitivity, inactive status), showed username, role, and password hash preview for successful query, and closed database connection after test completion

Database Setup and Initialization Script

Database Creation and Configuration (Datastore_Assembly.js)

MySQL2 promise library import — Imported MySQL2 with promise support to enable async/await database operations throughout the setup script.

File system operations import — Imported fs module to enable reading SQL schema files and writing configuration files during database setup.

Path utilities import — Imported path module to resolve file paths for SQL scripts and configuration files across different operating systems.

Terminal input interface import — Imported readline module to create interactive prompts for collecting database credentials and configuration during setup.

Environment variables loading — Configured dotenv to load existing environment variables and support environment-based configuration management.

Readline interface instance — Created readline interface instance to handle user input and output streams for interactive database setup prompts.

Asynchronous user input prompting — Implemented async wrapper function around readline's question method to enable promise-based user input collection.

Main setup orchestration function — Developed comprehensive setup function that coordinates database creation, table initialization, user creation, and initial data loading.

Script execution — Invoked main setup function to run the automated database installation and configuration process.

Database Purge Utility

Data Purge and Reset Utility (Purge_Actuator.js)

Environment configuration loading — Loaded database credentials and connection parameters from .env file using dotenv package for secure configuration management.

MySQL database connection setup — Created database connection instance using environment variables with fallback defaults for host, user, password, and database name.

Terminal input interface initialization — Configured readline interface for interactive command-line prompts to capture user confirmation before destructive operations.

Database connection establishment — Connected to MySQL database and handled connection errors with appropriate error messages and process termination.

User count retrieval — Queried the users table to count total number of user records before purge operation.

Feedback count retrieval — Queried the feedback table to count total number of feedback records before purge operation.

Data deletion confirmation prompt — Displayed current database contents and requested explicit "yes" confirmation from user before proceeding with irreversible data deletion.

Feedback data purge — Executed DELETE query to remove all feedback entries from the feedback table, deleted first due to foreign key constraints.

User data purge — Executed DELETE query to remove all user entries from the users table after feedback deletion completed.

Feedback table auto-increment reset — Ran ALTER TABLE command to reset the feedback table's auto-increment counter back to 1 for clean slate.

Users table auto-increment reset — Ran ALTER TABLE command to reset the users table's auto-increment counter back to 1 for clean slate.

Test Data Generation and Database Population (Simulation_Generator.js)

Environment configuration loading — Loaded database credentials and connection parameters from .env file using dotenv package for secure configuration management.

MySQL database connection setup — Created database connection instance using environment variables with fallback defaults for host, user, password, and database name.

Cryptography module initialization — Imported Node.js crypto module for AES-256-GCM email encryption functionality.

File system operations setup — Imported fs module to handle creation and management of image files in the uploads directory structure.

Path utilities initialization — Imported path module for cross-platform file path construction and management.

PNG image generation library — Imported pngjs library for programmatic creation of blank PNG image files.

Encryption key configuration — Retrieved AES-256 encryption key from environment variables for email field encryption.

Encryption algorithm specification — Defined AES-256-GCM as the encryption algorithm for securing email addresses in the database.

Email encryption function — Implemented AES-256-GCM encryption function that takes plaintext email and returns encrypted data with authentication tag and initialization vector.

Blank PNG image creation function — Created function to generate 400 x 300 pixel blank PNG files with text overlay indicating submission details and overlay theme.

First names array definition — Defined comprehensive array of realistic first names for generating diverse test user identities.

Last names array definition — Defined comprehensive array of realistic last names for generating diverse test user identities.

Sample pledge texts array — Created array of varied pledge commitment texts for testing feedback submission scenarios.

Data retention period options — Defined array of retention period values including "7days" and "indefinite" for testing data retention policies.

Learning responses array — Created array of sample responses for "what did you learn" question answers.

Topic interests array — Defined array of topic options for multiple-choice question answer testing.

Overlay theme names array — Created array of image overlay theme names including Nature, Ocean, Energy, Recycle, Tech, and Cute themes.

Random element selector utility — Implemented helper function to randomly select items from any provided array for generating varied test data.

Random name generator utility — Created function to combine random first and last names to generate full names for test users.

Random email generator utility — Implemented function to create email addresses by combining lowercased names with random email domains.

Random date generator utility — Created function to generate random dates within specified date ranges for realistic timestamp distribution.

Archive status calculator — Implemented function to determine whether feedback should be marked as archived based on submission date being older than 3 months.

Question IDs retrieval function — Created async function to fetch question IDs from database for proper answer record creation.

Directory creation utility — Implemented function to ensure uploads/photos and uploads/processed directories exist before generating images.

Main test data generation function — Created comprehensive async function to generate 500 test feedback entries organized into four distinct groups with specific date ranges, visit counts, and retention policies for testing cleanup functionality, advanced filters, and archive behavior.

Script execution logic — Validated encryption key length and format, then executed the main test data generation function to populate the database with realistic test data.

DATABASE

Database Schema and Structure

Database Schema Design and Implementation (schema.sql)

Complete database schema definition — Designed and implemented the full MySQL database structure for the DP Kiosk system, including all tables, relationships, indexes, and constraints

Users table structure — Created the users table with encrypted email storage, visit tracking, and automatic timestamp management for user registration and activity monitoring

Feedback table with retention controls — Designed the feedback table to store user submissions with support for photos, metadata, email tracking, admin notes, archival status, and configurable data retention periods

Questions and options system — Implemented a flexible question management system with the questions table supporting multiple question types (rating, text, choice), required field flags, display ordering, and active status controls

Question options relationship — Created the question_options table to support multiple-choice questions with ordered options linked to parent questions via foreign key constraints

Feedback answers tracking — Built the feedback_answers table to store user responses, linking submitted answers to both feedback entries and question definitions through cascading foreign key relationships

Admin users with role-based access — Designed the admin_users table with username, full name, password hash storage, role-based permissions (IT_staff, IT_admin, system_admin), login tracking, and account status management

Audit logging system — Created the audit_logs table to record administrative actions with details including action type, admin username, target information, IP address, user agent, and timestamp for compliance and security monitoring

Overlays theme management — Implemented the overlays table to manage photo overlay themes with separate desktop and mobile image paths, unique theme identifiers, display names, and ordering

Saved themes customization — Built the saved_themes table using JSON storage to allow admin users to create and save custom theme configurations with active status tracking and automatic timestamp updates

Database indexes optimization — Applied strategic indexes across all tables on foreign keys, timestamp fields, status flags, and frequently queried columns to improve query performance

Cascading delete relationships — Configured ON DELETE CASCADE foreign key constraints to maintain referential integrity when parent records are removed from users, questions, feedback, and admin_users tables

Default admin account initialization — Inserted the initial system administrator account with placeholder credentials for first-time setup and system access

Sample overlay themes data — Populated the overlays table with six default theme options (Nature, Ocean, Energy, Recycle, Tech, Cute) including file paths for both desktop and mobile versions

Sample questions seeding — Created initial demonstration questions including a rating question, text question, and multiple-choice question with four topic options to provide working examples

Database character set configuration — Applied utf8mb4 character set and unicode collation across all tables to support international characters and emoji storage

Table engine specification — Configured all tables to use InnoDB storage engine for transaction support, foreign key constraints, and crash recovery capabilities